OS 스케줄링 알고리즘을 활용한

다중 사용자 LLM API 요청 관리 시스템

졸업프로젝트 제안서

홍익대학교 컴퓨터공학과

C235180 서민지

지도교수: 이장호 교수님

2026년 3월

1. 서론

1.1 배경

최근 ChatGPT, Claude, Gemini 등 대규모 언어 모델(Large Language Model, LLM) API 서비스가 다양한 분야에서 활용되고 있다. 이에 따라 기업이나 연구실에서도 하나의 LLM API 계정을 여러 구성원이 공유하거나, 자체 LLM 서버를 구축하여 여러 사용자에게 서비스하는 사례를 찾아볼 수 있다.

이러한 다중 사용자 환경에서는 요청을 어떻게 관리할지가 문제가 된다. 주요 API 제공업체를 살펴보면, OpenAI는 조직(Organization) 단위로 분당 요청 수와 토큰 수에 한도를 두고 있고 [1], Anthropic도 사용 등급(Usage Tier)별로 호출 한도를 적용하고 있다 [2]. 그러나 이는 계정 단위의 관리이므로, 같은 계정을 공유하는 사용자들 사이에서 누가 얼마나 사용할지를 조율하는 것은 별개의 문제이다. 자체 LLM 서버를 운영하는 경우에도 마찬가지로, 사용자별 요청 관리 기능은 기본으로 제공되지 않는 것으로 보인다 (자세한 내용은 2장에서 살펴본다).

사용자별 관리 기능이 별도로 없다 보니, 요청은 결국 도착한 순서대로(First-Come, First-Served, FCFS) 처리되거나 호출 빈도 제한(Rate Limiting) 수준에서 관리되는 것으로 보인다. 이 경우 긴급한 요청도 순서를 기다려야 하고, 사용자별로 자원을 다르게 배분하기 어려우며, 자원이 공정하게 나누어지고 있는지 확인하기도 쉽지 않다.

1.2 동기 및 목적

3학년 운영체제 수업에서 프로세스 스케줄링 알고리즘을 배우면서, "이 알고리즘들을 LLM API 요청 관리에 적용해보면 어떨까?"라는 궁금증에서 이 프로젝트를 시작하게 되었다. 운영체제의 스케줄링 이론을 LLM API 환경에 적용해보고, 알고리즘별 성능과 공정성을 비교해보는 것이 본 프로젝트의 목적이다.

이 프로젝트에서는 구체적으로 다음 세 가지를 해보려고 한다.

1. **OS 스케줄링 알고리즘 적용**: FCFS, Priority Scheduling, MLFQ, WFQ 의 네 가지 대표적인 스케줄링 알고리즘을 LLM API 환경에 맞게 구현해본다.

2. **알고리즘별 성능 비교**: 평균 대기시간, 처리량 등의 지표로 각 알고리즘의 효율성을 비교해본다.
3. **공정성 분석**: Jain's Fairness Index(JFI)를 활용하여, 각 알고리즘이 사용자 간 자원을 얼마나 공정하게 배분하는지 분석해본다.

다음 장에서는 이들 알고리즘의 이론적 배경과 관련 도구를 살펴본다.

2. 관련 연구

이 장에서는 본 프로젝트에서 활용할 OS 스케줄링 알고리즘의 이론적 배경, 기존 LLM 서빙 도구의 현황, 그리고 공정성 측정 방법을 살펴본다.

2.1 OS 스케줄링 알고리즘

프로세스 스케줄링은 운영체제의 핵심 기능 중 하나로, CPU 자원을 여러 프로세스에 배분하기 위한 다양한 알고리즘이 있다 [3][4].

FCFS(First-Come, First-Served)는 요청이 도착한 순서대로 처리하는 가장 단순한 방식이다. 구현이 간단하지만, 처리 시간이 긴 요청 때문에 짧은 요청들까지 지연되는 호위 효과(Convoy Effect)가 발생할 수 있다 [3].

Priority Scheduling은 각 프로세스에 우선순위를 부여하여 우선순위가 높은 프로세스를 먼저 처리한다. 다만 우선순위가 낮은 프로세스가 계속 밀려 처리되지 못하는 기아 현상이 발생할 수 있으며, 이를 방지하기 위해 오래 기다린 프로세스의 우선순위를 점진적으로 높여주는 에이징 기법이 사용된다 [3].

MLFQ(Multi-Level Feedback Queue)는 여러 개의 큐를 두고, 작업의 실행 특성에 따라 우선순위를 동적으로 조정하는 알고리즘이다. 짧은 작업은 높은 우선순위 큐에서 빠르게 처리되고, 타임 퀀텀(Time Quantum, 한 번에 할당되는 최대 실행 시간)을 초과한 긴 작업은 점차 하위 큐로 이동한다. OS에서 MLFQ는 선점형으로 동작하여, 타임 퀀텀을 초과한 프로세스를 중단하고 하위 큐로 강등시킨다 [4].

WFQ(Weighted Fair Queuing)는 네트워크 분야에서 사용되는 공정 큐잉 알고리즘이다 [5]. 각 흐름(flow)에 가중치를 부여하고, 패킷마다 가상 완료 시각(Virtual Finish Time)을 계산하여

이 값이 가장 작은 패킷을 먼저 전송한다. 가상 완료 시각은 처리 비용을 가중치로 나누어 구하기 때문에, 가중치가 큰 흐름일수록 값이 작아져 더 자주 선택된다. 이론적으로는 GPS(Generalized Processor Sharing)라는 이상적인 모델이 있지만 실제로 구현이 어려워, 이에 가깝게 구현한 것이 WFQ이다.

2.2 LLM 서빙과 스케줄링

LLM 서빙(LLM을 사용자에게 제공하는 것) 분야에서 가장 널리 사용되는 오픈소스 도구를 조사해보았다.

vLLM은 UC Berkeley에서 개발한 LLM 서빙 도구로, PagedAttention이라는 메모리 관리 기법으로 높은 처리 성능을 보여준다 [6]. 다만 공식 문서와 소스 코드를 조사한 결과, 요청 스케줄링은 선착순 방식에 가까우며 사용자 간 공정성을 보장하는 별도의 기능은 확인하지 못했다.

Hugging Face TGI는 오픈소스 LLM 서빙 도구로, 다양한 LLM을 쉽게 배포할 수 있도록 도와준다 [7]. 그러나 공식 문서에서 다중 사용자 환경에서의 요청 우선순위 관리나 공정성 보장 기능은 찾아볼 수 없었다.

조사한 도구들은 주로 응답 속도를 높이는 데 집중하고 있었으며, 다중 사용자 환경에서의 요청 스케줄링이나 사용자 간 공정성 문제는 상대적으로 덜 다루어진 것으로 보인다. 본 프로젝트는 이 부분에 OS 스케줄링 이론을 적용해보려고 한다.

2.3 공정성 측정

여러 사용자가 자원을 공유하는 시스템에서, 자원 배분의 공정성을 측정하는 지표로 Jain's Fairness Index(JFI)가 있다 [5]. JFI는 네트워크 대역폭 배분, CPU 스케줄링 등 자원 공유 문제에서 사용되는 지표로, 0에서 1 사이의 값 하나로 공정성을 직관적으로 나타낼 수 있어 본 프로젝트의 알고리즘 비교에 적합할 것으로 보았다.

$$JFI = (\Sigma x_i)^2 / (n \cdot \Sigma x_i^2)$$

여기서 x_i는 사용자 i가 할당받은 자원량이며, n은 사용자 수이다. 본 프로젝트에서도 이 지표를 활용하여 스케줄링 알고리즘의 공정성을 측정해볼 계획이다. 다음 장에서는 이들 알고리즘과 JFI를 활용한 제안 시스템의 구조를 설명한다.

3. 제안 시스템

3.1 시스템 개요

본 프로젝트에서는 OS 스케줄링 알고리즘을 LLM API 요청 관리에 적용한 시스템을 구현해 보려고 한다. 기본 아이디어는, OS에서 프로세스가 CPU를 할당받기 위해 대기하는 것처럼, LLM API 요청도 대기열에서 순서를 기다리는 구조를 만드는 것이다. 표 1은 OS 개념과 LLM 환경 간의 대응 관계를 정리한 것이다.

표 1. OS 개념과 LLM 환경 대응 관계

OS 개념	LLM 환경 대응	설명
프로세스 (Process)	LLM API 요청	스케줄링의 기본 단위
CPU 시간 (CPU Time)	LLM 추론 시간	할당되는 자원
준비 큐 (Ready Queue)	요청 대기열	처리를 기다리는 작업의 저장소
우선순위 (Priority)	사용자 등급, 요청 긴급도	처리 순서 결정 기준

시스템의 전체 흐름은 다음과 같다. 사용자가 REST API를 통해 요청을 보내면, 먼저 호출 빈도 제한(Rate Limiting)으로 과도한 요청을 걸러낸 뒤 대기열에 등록한다. 스케줄러는 선택된 알고리즘에 따라 대기열에서 다음에 처리할 요청을 결정하고, 해당 요청을 LLM API에 전달한다. 응답이 돌아오면 사용자에게 결과를 전달하고, 스케줄러는 다음 요청을 꺼내 같은 과정을 반복한다.

이처럼 호출 빈도 제한은 요청의 진입을 관리하는 전처리 단계이고, 사용자 간 자원 배분은 스케줄러가 담당한다. 시스템은 요청 수신부(REST API), 대기열, 스케줄러, LLM 연결부의 네 부분으로 구성하며, Node.js와 Express.js를 사용하여 구현할 계획이다.

스케줄링 알고리즘은 교체 가능하도록 설계하여, 스케줄러만 바꾸면 나머지 부분은 그대로 유지한 채 네 가지 알고리즘을 같은 조건에서 비교할 수 있게 하려고 한다.

3.2 설계 방침

OS 스케줄링 알고리즘을 LLM 환경에 그대로 적용할 수는 없으므로, 본 프로젝트의 핵심인 스케줄러를 중심으로 다음과 같은 설계 방침을 정했다.

첫째, 모든 알고리즘을 **비선점형** 방식으로 설계한다. 2장에서 살펴본 것처럼 OS에서 MLFQ 등은 선점형으로 동작하여 실행 중인 프로세스를 중단할 수 있지만, LLM 추론은 한번 시작되면 중간에 중단하기 어렵기 때문이다. 따라서 스케줄러는 대기열에서 다음 요청을 선택하는 시점에만 알고리즘을 적용하고, 이미 처리 중인 요청은 완료될 때까지 기다린다.

둘째, 각 알고리즘의 **핵심 원리는 그대로 유지하면서, 스케줄링 기준은 LLM 환경에 맞게 조정**한다. FCFS는 별도의 변형 없이 그대로 적용하지만, 나머지 세 알고리즘은 다음과 같이 조정한다.

- **Priority Scheduling**: 사용자 등급(예: 일반, 프리미엄)을 우선순위로, 대기 시간을 에이징 기준으로 사용한다.
- **MLFQ**: 타임 퀀텀 초과 대신 해당 사용자의 최근 요청들의 평균 추론 시간을 기준으로 큐를 재배정하여, 짧은 요청이 빠르게 처리될 수 있도록 한다.
- **WFQ**: 사용자별 가중치에 따라 가상 완료 시각을 계산하여, 가중치에 비례한 서비스를 제공하도록 설계한다.

3.3 평가 계획

구현한 네 가지 알고리즘의 성능과 공정성을 비교하기 위해 평균 대기시간, 처리량, 공정성 지수(JFI)를 측정 지표로 사용한다.

실험은 여러 사용자가 동시에 요청을 보내는 상황을 만들어 진행할 계획이다. 테스트 스크립트를 작성하여 사용자 수(예: 3명, 5명, 10명), 요청 빈도(저부하, 고부하), LLM API 동시 요청 수 등을 조절하면서 다양한 부하 시나리오를 재현해보려고 한다. 또한 사용자별 요청 패턴을 달리하여(예: 짧은 질문 위주 사용자와 긴 문서 요청 사용자의 혼합), 알고리즘별 반응 차이도 관찰할 계획이다.

FCFS를 기준(베이스라인)으로 삼고, 나머지 세 알고리즘을 같은 조건에서 실행한 뒤 위 세 가지 지표로 결과를 비교하여, 각 알고리즘이 어떤 상황에서 유리하고 어떤 한계가 있는지를 파악해보려고 한다.

3.4 기대 효과

본 프로젝트를 통해 다음과 같은 점들을 확인해볼 수 있을 것이다.

- **등급별 서비스와 공정성 비교**: Priority Scheduling 이나 WFQ 처럼 사용자 등급에 따라 자원을 다르게 배분하는 방식과, JFI 로 측정되는 균등한 배분은 동시에 만족시키기 어려운 면이 있다. 같은 조건에서 두 가지를 함께 비교하여, 어떤 상황에서 어떤 방식이 더 적합한지 살펴보려고 한다.
- **다른 서비스에도 적용할 수 있는 구조**: 본 시스템의 대기열-스케줄러 구조는 LLM API 에만 쓸 수 있는 것이 아니라, 대기열과 우선순위 관리가 필요한 다른 API 서비스에도 비슷하게 적용해볼 수 있을 것이다.
- **OS 이론이 실제로 유용한지 확인**: 교과서에서 다루는 스케줄링 알고리즘이 실제 서비스 환경에서도 의미 있는 성능 차이를 만들어내는지 실험을 통해 확인해볼 수 있다.

다음 장에서는 이러한 목표를 달성하기 위한 구체적인 연구 일정을 정리한다.

4. 연구 일정

3장에서 설명한 시스템을 한 학기 동안 구현하고 실험까지 마무리하는 것을 목표로 한다. 시스템 구현이 완료되어야 실험을 진행할 수 있으므로, 4월까지 구현을 마치고 5월에 실험과 분석에 집중하는 일정을 계획하였다.

표 2. 2026년 1학기 연구 일정

기간	주요 활동	산출물	마감일
3월	관련연구 조사, 제안서 작성	제안서 (본 문서)	3/22
4월 초	시스템 설계, 기본 구조 구현	중간보고서	4/12
4월 중~말	스케줄링 알고리즘 구현 및 단위 테스트	동작 가능한 프로토타입	4/30
5월 초~중	실험 수행 및 결과 분석	최종보고서 + 소스코드	5/24
5월 말	발표 자료 준비, 시스템 데모 구성	발표 PPT, 데모	5/26~29

이상으로 본 프로젝트의 배경, 관련 연구, 시스템 설계 방향, 그리고 연구 일정을 정리하였다. 위 일정에 따라 네 가지 스케줄링 알고리즘을 구현하고 성능과 공정성을 비교하여, OS 스케줄링 이론이 LLM API 요청 관리에서 실제로 의미 있는 차이를 만들 수 있는지 확인해보려고 한다.

참고문헌

[1] OpenAI, "Rate limits," OpenAI API Documentation. [온라인] Available: https://platform.openai.com/docs/guides/rate-limits (접속: 2026-03-14)

[2] Anthropic, "Rate limits," Anthropic API Documentation. [온라인] Available: https://docs.anthropic.com/en/api/rate-limits (접속: 2026-03-14)

[3] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018. [온라인] Available: https://www.os-book.com/

[4] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Version 1.10, Arpaci-Dusseau Books, 2023. [온라인] Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

[5] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 9th ed., Pearson, 2025. [온라인] Available: https://gaia.cs.umass.edu/kurose_ross/

[6] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in Proc. 29th ACM Symposium on Operating Systems Principles (SOSP '23), 2023. [온라인] Available: https://arxiv.org/abs/2309.06180

[7] Hugging Face, "Text Generation Inference (TGI)." [온라인] Available: https://huggingface.co/docs/text-generation-inference/ (접속: 2026-03-14)